RE Meet 4

Learning Regular Sets from Queries and Counterexamples*

-Dana Angluin

Intro

- A learning algorithm L* is described that correctly learns any regular set from any minimally adequate Teacher in polynomial time in the number of states of the minimum dfa for the set and the maximum length of any counterexample provided by the Teacher.
- When we teach someone, we don't just give them random data. We provide “crucial” examples and correct their mistakes to help them learn faster. This is the core idea behind the Learner and Teacher model. The learning process isn't passive (just looking at a pile of data); it's an active dialogue where the Learner asks targeted questions to a “helpful” teacher to build its understanding of the system's rules.
- It is shown that in a stochastic setting the ability of the Teacher to test conjectures may be replaced by a random sampling oracle, EX( ).
  - Meaning the powerful (and somewhat unrealistic) ability to test for equivalence can be replaced by a random sampling oracle in a more practical, stochastic setting.

EX() Random Sampling Oracle

When the Learner pushes the button, the oracle does two things:

- It picks a random string x from the universe of all possible strings, based on some underlying probability distribution.
- It tells the Learner whether that string x is a member of the unknown language L (giving a "yes" or "no" label). Essentially, it provides a randomly generated, correctly labeled example.

  Instead of asking a Teacher for a perfect proof of correctness since this Equivalence Query is a very strong assumption and it requires the Teacher to have a perfect model and a way to compare it.

  Instead the Learner tests its guessed automaton against a large number of these random samples . A mismatch provides a counterexample, while a lack of mismatches gives high statistical confidence that the learned model is "approximately correct" .

Minimally Adequate Teachers

The Teacher's Two Core Abilities:

- Membership Queries: The Learner can provide any string **t** and ask the Teacher if that string is a member of the unknown set **U**. The Teacher must provide a correct "**yes**" or "**no**" answer . This is the most basic tool the Learner has for gathering specific facts.
- Equivalence Queries (Conjectures): After gathering enough information, the Learner can form a complete hypothesis by constructing a DFA. It then presents this DFA to the Teacher as a conjecture. The Teacher's job is to check if the language accepted by this DFA is exactly the same as the unknown language **U**:
    - if the Learner's guess is correct, the Teacher replies "**yes**," and the learning process is complete.
    - If the guess is incorrect, the Teacher must reply with a **counterexample**. This is a specific string that proves the Learner's DFA is wrong. The counterexample will be a string that is either in **U** but rejected by the Learner's DFA, or not in **U** but accepted by the Learner's DFA

How "Reasonable" is the Teacher?

- Membership Queries are **Realistic**:
  - The paper argues that the ability to classify a single example (a membership query) is a very reasonable and "unobjectionable" capability to expect from any expert or system you're trying to model. It's the most basic form of interaction.
- Equivalence Queries are "**Problematic**":
  - The paper frankly admits that the equivalence query is a much stronger and more questionable assumption. It seems to require that the Teacher:
    - Already has a perfect, explicit representation of the correct model.
    - Can perform what might be a very complex computation to test if the Learner's guessed DFA is equivalent to its own.

To soften this strong assumption, the paper points out that this perfect equivalence oracle can be replaced by a more practical random sampling oracle in a real-world setting.

This means instead of asking "Is my model perfect?", you ask "**Does my model work for a large number of random examples?**".

How Feasible is the Teacher's Job?

Can the teacher perform its job efficiently?

The answer is yes, under the right conditions. If we assume the unknown language is represented by a DFA with n states, then the Teacher's job is computationally easy :

- **Answering Membership Queries:** Simply involves running the given string on the DFA, which is very fast.
- **Answering Equivalence Queries:** Involves running a standard, polynomial-time algorithm for testing if two DFAs are equivalent. This algorithm also conveniently provides a counterexample if they are not.

This shows that the model doesn't hide an impossibly difficult computation on the Teacher's side.

Related Results - Solving a Hard Problem

- **The Challenge: A Computationally Hard Problem**
  - Prior work (by E. M. Gold) proved that finding the smallest DFA from a static set of positive and negative examples is **NP-hard** .
  - This means there is no known efficient algorithm, the problem is considered computationally intractable.
- **The Breakthrough: The Power of Active Queries**
  - Angluin's key insight is that this hard problem becomes efficiently (polynomially) solvable if the Learner can **actively ask membership queries** .
  - This "active learning" approach avoids the intractable search by allowing the algorithm to gather specific, targeted information as needed.

The Observation Table

The Learner organizes everything it learns from **Membership Queries** into a data structure called an **Observation Table**.

It consists of three parts: `(S, E, T)`.

- **S (State Prefixes)**: A set of strings representing paths from the start state. These are the candidates for the states of the DFA we're building. This set is always **prefix-closed**

  (if `"ab"` is in S, `"a"` and `""` must also be).

- **E (Experiment Suffixes)**: A set of strings used to test and distinguish the states in S. They are the "experiments" we run from each potential state. This set is **suffix-closed**.

- **T (The Data)**: A function that stores the results.
  - `T(s·e)` is 1 if the string `se` is in the language, and `0` otherwise.

The row(s) Fingerprint

- The key concept is the **`row(s)`**, which is the sequence of 0s and 1s for a prefix `s` across all experiments in `E`.
- We can think of `row(s)` as the **"behavioral fingerprint"** of the state reached by `s`.
- If two prefixes s_1 and s_2 have the same fingerprint ($row(s_1) = row(s_2)$), they are currently indistinguishable.
- The table has two sections of rows:
  - **Upper Section**: Rows for prefixes in `S` (our current states).
  - **Lower Section**: Rows for one-letter extensions, $S \cdot A$ (used to build transitions).

    Where A is the alphabet set like {0,1} or {a,b}

When is the Table "Stable"? Two Golden Rules

Before the Learner can build a DFA, the table must be **closed** and **consistent**.

- **Closed Table**
  - **Question**: Does every behavior we can reach in one step (from rows in $S \cdot A$) match a behavior we already know (from a row in S)?
  - **Meaning**: We haven't discovered any brand-new, unclassified state behaviors.
  - For each *t* in S.A there exists an *s* in S s.t. row(*t*)=row(*s*).

- **Consistent Table**
  - **Question**: If two prefixes s_1 and s_2 seem to be the same state ($row(s_1) = row(s_2)$), do they behave identically after any next character a?
  - **Meaning**: Our "fingerprints" are reliable. If two states look the same, they truly are the same for all one-step transitions.
  - If row(s1) = row(s2) then for a in A, row(s1.a)=row(s2.a)

Building the DFA from a Stable Table

- Once the table is closed and consistent, the Learner can build a hypothesis DFA, `M(S,E,T)`.
- **States Q**: The set of unique "fingerprints" (`row(s)`) from the set S.
- **Initial State q_0**: The fingerprint of the empty string, `row(ε)`.
- **Final States F**: Any state `row(s)` where `T(s)` is 1 (i.e., the prefix itself is an accepting string).
- **Transitions δ**: The transition from a state `row(s)` on a character a is the state represented by the fingerprint `row(s·a)`.

$$Q = \{\mathrm{row}(s) \colon s \in S\},$$

$$q_0 = \mathrm{row}(\lambda),$$

$$F = \{\mathrm{row}(s) \colon s \in S \text{ and } T(s) = 1\},$$

$$\delta(\mathrm{row}(s), a) = \mathrm{row}(s \cdot a).$$

Table to Automaton

- The paper first confirms that the process of building a DFA from a **closed and consistent** table is mathematically sound.
- **Consistency is Key for Transitions:** The `consistent` property guarantees that if two prefixes s_1 and s_2 represent the same state ($row(s_1) = row(s_2)$), their transitions on any character a will also lead to the same state ($row(s_1 \cdot a) = row(s_2 \cdot a)$). This ensures the transition function δ is well-defined.
- **Closedness is Key for States:** The `closed` property guarantees that for any state $row(s)$ and character a, the resulting fingerprint $row(s \cdot a)$ is already represented by a known state in our state set Q. The automaton never transitions to an unknown state.

Theorem 1

This theorem is the most important result on this page. It makes two powerful claims about the DFA, `M(S, E, T)`, that we build from a stable table:

1. **It's Correct on Known Data:** The automaton is **consistent with the finite function T**. This means the DFA will correctly accept or reject every string that the Learner has already asked about and stored in its table.
2. **It's the Smallest Possible Model:** Any other DFA that is *also* consistent with the table T but is different from our `M` **must have more states**. This proves that the L* algorithm constructs the unique, minimal DFA that can explain all the observations made so far.

The Proof's First Step - Lemma 2

The paper proves Theorem 1 using a series of smaller, supporting lemmas.
Lemma 2 : establishes the fundamental connection between a string (a path) and a state (a fingerprint) in the constructed automaton `M`.
What it says: "For any prefix `s` (from `S` or `S · A`), if you trace the path for `s` from the initial state of the DFA, you are guaranteed to end up in the state represented by the fingerprint `row(s)`."
How it's proven: Using a straightforward induction on the length of the string `s`.

$$
\begin{aligned}
\delta(q_0, t) &= \delta(\delta(q_0, s), a) \\
&= \delta(\mathrm{row}(s), a), && \text{by the induction hypothesis,} \\
&= \mathrm{row}(s \cdot a), && \text{by the definition of } \delta, \\
&= \mathrm{row}(t), && \text{since } t = s \cdot a.
\end{aligned}
$$

The Proof's Second Step (Lemma 3)

- **Goal**: To prove that the constructed DFA `M` is fully **consistent with the entire observation table T**.
- **The Claim**:
  - For *every* prefix s in $(S \cup S \cdot A)$ and *every* experiment e in E:
  - The DFA `M` will accept the string $s \cdot e$ if and only if the table entry $T(s \cdot e)$ is 1.
- **In Simple Terms**:
  - This lemma confirms that our DFA doesn't just get the state prefixes right; it correctly handles **all combinations of prefixes and experiments** that we have information for.
  - It proves that the model perfectly matches all of our observations.
- **How It's Proven**:
  - The paper proves this using **induction on the length of the experiment string e**.
  - **Base Case (e = ε)**: This is shown to be true directly from the definition of Final States (F) in the DFA.
  - **Inductive Step**: It assumes the claim is true for all shorter experiments and then uses **Lemma 2** and the **closed** property of the table to show it must also be true for longer experiments.

$$
\begin{aligned}
\delta(q_0, s \cdot e) &= \delta(\delta(q_0, s), a \cdot e_1) \\
&= \delta(\mathrm{row}(s), a \cdot e_1), && \text{by the preceding lemma,} \\
&= \delta(\mathrm{row}(s_1), a \cdot e_1), && \text{since } \mathrm{row}(s) = \mathrm{row}(s_1), \\
&= \delta(\delta(\mathrm{row}(s_1), a), e_1) \\
&= \delta(\mathrm{row}(s_1 \cdot a), e_1), && \text{by the definition of } \delta, \\
&= \delta(\delta(q_0, s_1 \cdot a), e_1), && \text{by the preceding lemma,} \\
&= \delta(q_0, s_1 \cdot a \cdot e_1).
\end{aligned}
$$

By the induction hypothesis on e_1, $\delta(q_0, s_1 \cdot a \cdot e_1)$ is in F if and only if $T(s_1 \cdot a \cdot e_1) = 1$. Since $\mathrm{row}(s) = \mathrm{row}(s_1)$ and $a \cdot e_1 = e$ is in E, $T(s_1 \cdot a \cdot e_1) = T(s \cdot a \cdot e_1) = T(s \cdot e)$. Hence $\delta(q_0, s \cdot e)$ is in F and only if $T(s \cdot e) = 1$, as claimed.

Lemma 4 - Our DFA is the Unique Smallest Model

- **The Claim**: To prove that the DFA `M` we built is not just correct for our data, but is also the **smallest and only** DFA of its size that can be.
- **The Guarantee**: This is the crucial result that ensures the L* algorithm produces the **unique minimal DFA** for the data it has collected. No smaller model, and no different model of the same size, can explain the observations in the table.
- **Proof Sketch**: The proof confirms this by showing that any other consistent DFA must have at least as many states as ours. It then defines a one-to-one mapping between the states and verifies that this mapping preserves the entire DFA structure (initial state, transitions, and final states).

The Leaner L*

1. **Initialize**: The Learner starts with a tiny observation table where $S = \{\varepsilon\}$ and $E = \{\varepsilon\}$. It asks membership queries to fill this initial table .

2. **Stabilize the Table (Inner Loop)**: The Learner repeatedly checks the table:
   - **If Not Consistent**: It finds the inconsistency and adds a new **experiment (suffix)** to E to distinguish the rows. It then asks more membership queries to fill the new column .
   - **If Not Closed**: It finds a new behavior and adds the corresponding **prefix** to S. It then asks more membership queries to fill the new rows .
   - This loop continues until the table is both **closed and consistent**.

3. **Make a Conjecture**: Once the table is stable, the Learner constructs the DFA $M(S,E,T)$ and asks the Teacher an **Equivalence Query** .

4. **Process the Teacher's Feedback**:
   - If the Teacher replies **"yes"**: The algorithm halts and outputs the correct DFA .
   - If the Teacher replies with a **counterexample** t: The Learner adds t and all of its prefixes to S, updates the table with new membership queries, and goes back to Step 2 to re-stabilize its now larger and more informed table .

Why the Algorithm is Correct

- The paper briefly addresses the algorithm's correctness, which is quite straightforward .
- The algorithm only terminates when the Teacher gives a "yes" answer to an equivalence query.
- Since the Teacher is assumed to be "minimally adequate" (i.e., always truthful), a "yes" answer guarantees that the Learner's final output is a correct DFA for the unknown language.

The Termination Proof - Why L* Always Finishes

- **The Question**: How do we know the L* algorithm won't get stuck in an infinite loop of fixing its table and making bad guesses?
- **The Answer**: The algorithm makes guaranteed progress in two ways:
  1. The **Observation Table** consistently grows in a structured way.
  2. The number of **states in its conjectured DFAs** is strictly increasing.
- Both of these are bounded by n, the number of states in the true minimal DFA.

The Proof's Foundation (Lemma 5)

The Claim: The number of unique "fingerprints" (`row(s)`) in our table's S section is a lower bound on the number of states in *any* DFA that is consistent with our observations .
Example: If our table has 5 unique fingerprints, any correct DFA must have at least 5 states to represent those distinct behaviors.
Significance: This lemma connects the size of our table to the size of the final DFA we are trying to learn. It proves that the number of unique rows we find can never exceed n, the number of states in the true minimal DFA.

Guarantee 1: The Table Stabilization Loop is Finite

Every time the algorithm fixes the table, it makes measurable progress.

- If the table is **not consistent**, fixing it splits one row into two, increasing the number of unique rows .
- If the table is **not closed**, the algorithm adds a new prefix with a brand-new fingerprint, increasing the number of unique rows .

Conclusion: Since the number of unique rows must be less than or equal to n, this stabilization process can only happen a finite number of times before the algorithm must make a conjecture, which is at most **(n-1)** times since initially at least one value of row(s) and there cannot be more than n.

Guarantee 2: The Number of Guesses is Finite

- The number of states in a conjectured DFA is equal to the number of unique rows in S.
- When the Teacher provides a counterexample, the algorithm adds new information to S. The proof shows that this new information will force the next stable table to have *more* unique rows .
- **Conclusion**: The number of states in the sequence of incorrect DFAs is strictly increasing and is bounded by n. Therefore, the Learner can make at most **n** total conjectures before finding the correct one .

In short: L* correctly terminates after making at most n conjectures and executing its main loop at a total of at most (n-1) times.

Time analysis, The Key Parameters:

- To analyze the algorithm's efficiency, the paper measures its work in terms of three key variables:
  - **`n`**: The number of states in the minimal DFA for the unknown language. This represents the complexity of the target.
  - **`m`**: The maximum length of any counterexample provided by the Teacher. This represents the complexity of the feedback.
  - **`k`**: The size of the alphabet `A`.
- The goal is to show that the total running time is a **polynomial function** of `m` and `n`.

Bounding the Observation Table's Size

The analysis works by first finding upper bounds on the size of the observation table.

Bounding the Columns (E):

- The set `E` only grows when the table is found to be **not consistent**.
- Fixing an inconsistency always increases the number of unique `row(s)` fingerprints.
- The number of unique fingerprints is capped by `n`.
- **Conclusion**: `|E|` can be at most `n` .

Bounding the Rows (S):

- The set `S` grows when the table is **not closed** (at most `n-1` times) or when a **counterexample** is processed (at most `n-1` times).
- Each counterexample (max length `m`) adds at most `m` prefixes to `S`.
- **Conclusion**: `|S|` is bounded by `n + m(n-1)`, which is `O(mn)`.

Bounding the String Lengths:

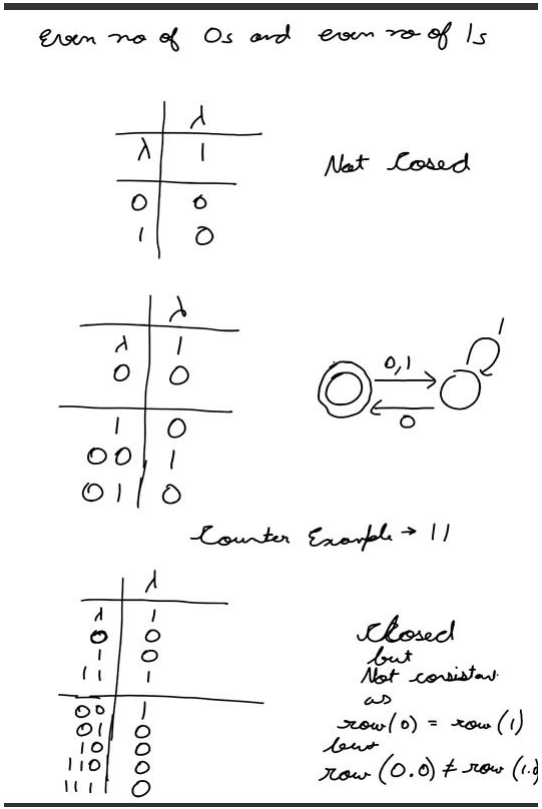
- The maximum length of any prefix in S is bounded by $m + n - 1$.
- The maximum length of any experiment in E is bounded by $n - 1$.
- Therefore, the maximum length of any string the Learner queries $(s \cdot a \cdot e)$ is $O(m+n)$.

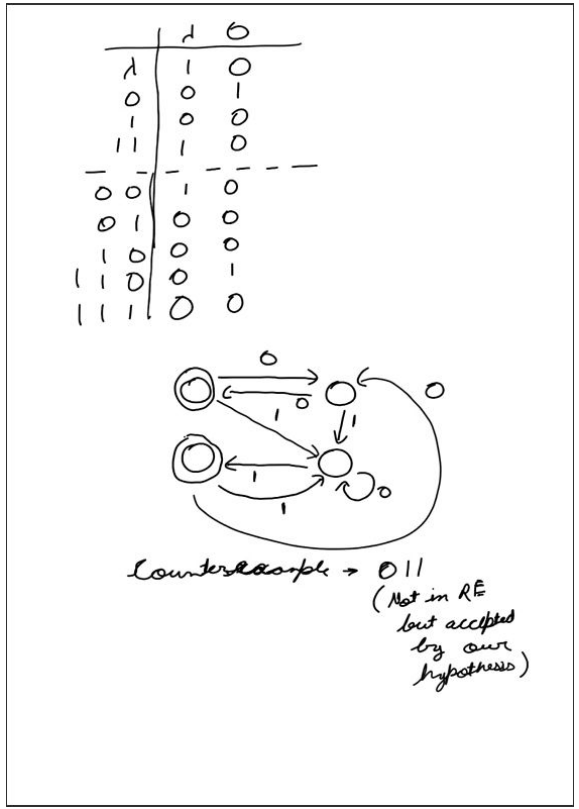
Total Table Size:

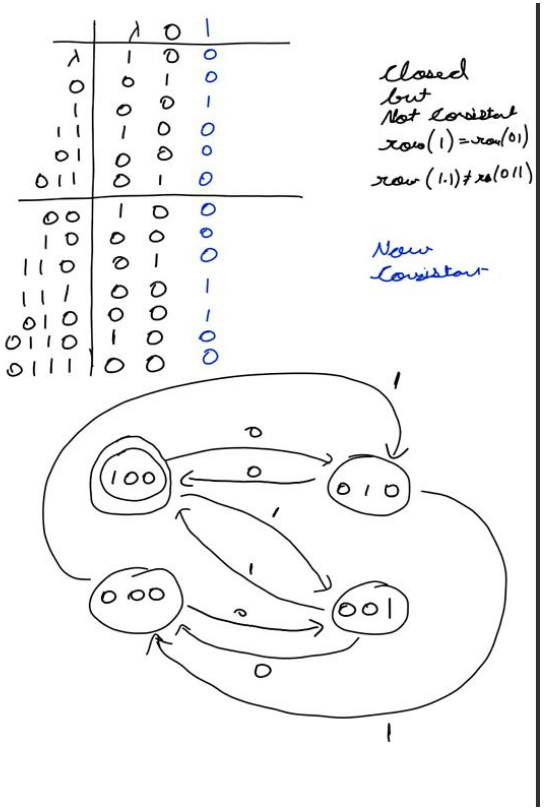
- The table has $|S \cup S \cdot A|$ rows, which is $O(kmn)$.
- With $O(n)$ columns, the total size of the observation table is **polynomial** in m and n(considerign $k<<m$ and $k<<n$)

Conclusion: The total running time of the L* algorithm is bounded by a **polynomial function of m and n**.

Example Run For L*







One problem is to find a method to reduce substantially the number of membership queries.

Finding Counter-Example Using Sampling

The requirements for the "minimally adequate Teacher" are so strong that they don't apply to most real-world learning problems.

Computational Hardness: For many complex problems, it's computationally impossible or extremely difficult to definitively test if two hypotheses are equivalent. Finding a counterexample for a general hypothesis can be an NP-hard problem.

Need for a Perfect Oracle: The MAT framework assumes the Teacher has perfect, complete knowledge of the "correct" hypothesis. In reality, data is often noisy, incomplete, or based on complex, unknown distributions. A system that can always provide a perfect counterexample simply doesn't exist for many domains.

As a solution, it proposes a "stochastic setting" (incorporates randomness and probability) based on the work of Leslie Valiant, where the Teacher's role is simplified. Instead of giving definitive answers and counterexamples, the Learner uses a **random sampling oracle** to get random examples. The goal shifts from finding a perfectly correct hypothesis to finding one that is **approximately correct with high probability**.

The Stochastic Setting

In computational theory, an **oracle** is a theoretical "black box" that can solve a specific problem instantly and with perfect accuracy. It is a concept used to study the limits of what a computer can do.
Eg- Kind of like a magic 8-ball🎱

Assume some probability distribution Pr (not known to learner) on set of all strings over the alphabet A. The learner has access to information about U by means of two oracles-

- Membership Oracle- **Member(x)**
  Returns yes if x is an element of U(language) and no otherwise.
- Random Sampling Oracle- **EX()**
  It selects a string x from A* according to the distribution Pr and returns the pair (x,d), where d is yes if x is in U and otherwise no. Separate calls to EX() are statistically independent events.

In addition, events. the Learner is given, at the start of the computation, two positive numbers between 0 and 1, the accuracy ε and the confidence δ.

Approximation

Let ε denote a positive number between 0 and 1, and let V and W denote sets of strings over the alphabet A. V is an ε-approximation that

$$\sum_{x \in V \oplus W} Pr(x) \leqslant \varepsilon.$$

($V \oplus W$ denotes the symmetric difference of V and W, that is, the elements that are in exactly one of the two sets). If M is a dfa, it is said to be an ε-approximation of the set W provided the set of strings V accepted by M is an ε-approximation of W.

If M is an ε-approximation of U, then the probability of finding a discrepancy between the set accepted by M and U with one call to the random sampling oracle EX() is at most ε.

The Approximate Learner La*(Modified L*)

A membership query of the sting x is satisfied by a call to Member(x).

Each conjecture is tested be a number of calls to EX(). If any of the calls returns a pair (t,d) such that d=yes but M rejects it or vice versa, then t is said to be the counterexample and proceeds to modify its hypothesis just like L*. If none of the calls returns a counterexample, it halts with output M. The number of calls depend on the accuracy ε and confidence δ parameters, as well as how many previous conjectures it has tested.

$$r_i = \frac{1}{\varepsilon}\left(\log\frac{1}{\delta} + (\log 2)(i+1)\right).$$

If i previous conjectures have been tested, then L_a^* makes $\lceil r_i \rceil$ calls to $EX(\ )$. (Note: log denotes the natural logarithm.)

The main result concerning L_a^* is the following.

δ	0	1
q_0	q_1	q_2
q_1	q_0	q_3
q_2	q_3	q_0
q_3	q_2	q_1

THEOREM 7. *If n is the number of states in the minimum dfa for the unknown regular set U, then L_a^* terminates after $O(n+(1/\varepsilon)(n\log(1/\delta)+n^2))$ calls to the $EX(\ )$ oracle. Moreover, the probability that the acceptor output by L_a^* is an ε-approximation of U is at least $1-\delta$.*

To see that this holds, note that as before there will be a total of at most $n-1$ counterexamples. The total number of calls to $EX(\ )$ before L_a^* terminates is at most

$$\sum_{i=0}^{n-2}(r_i+1)=(n-1)+\frac{1}{\varepsilon}\left((n-1)\log\left(\frac{1}{\delta}\right)+(\log 2)\sum_{i=0}^{n-2}(i+1)\right)$$
$$=O\left(n+\frac{1}{\varepsilon}\left(n\log\left(\frac{1}{\delta}\right)+n^2\right)\right).$$

What is the probability that L_a^* will terminate with an acceptor that is not an ε-approximation of U after testing i previous conjectures? It is at most $(1-\varepsilon)^{r_i}$, so the probability that L_a^* terminates with an acceptor that is not an ε-approximation of U is at most

$$\sum_{i=0}^{n-2}(1-\varepsilon)^{r_i}\leqslant\sum_{i=0}^{n-2}e^{-\varepsilon r_i}$$
$$\leqslant\sum_{i=0}^{n-2}\frac{\delta}{2^{i+1}}$$
$$\leqslant\delta.$$

This concludes the proof of Theorem 7.

Running Time of La*-
If n is the number of states of minimum dfa for U and m is an upper bound on the length of any string returned by EX() during the run, then the running time is bounded by a polynomial in m and n. If the distribution Pr is such that it concentrates a lot of probability on very long strings (compared to n), then this is not much of a guarantee.

We can do preliminary sampling from Ex() to determine a cutoff length l such that with probability at least (1-δ/2) the weight of the strings longer than l is at most ε/2. Then La* runs as before replacing the parameter by their new values and ignoring any strings that are longer than l.

approximation of U. The termination claim and argument must be modified somewhat to account for the (unlikely) possibility that $EX(\ )$ will return many strings of length greater than l, and of which are ignored. Standard statistical analysis shows that the number of samples that L_a^* must take in the preliminary phase is bounded by a polynomial in $1/\varepsilon$ and $1/\delta$.

Context-Free Grammars (Special Case)

Assumes it also answers “existential queries” and “termination queries”. Now, we describe a Learner L^{cf} for a class of context free languages. There is an unknown context free grammar G. The learner knowns the set of terminal symbols T, the set of non-terminal symbols V and the start symbol S of G. The queries-

- Membership query - MEMBER(x,A) where x is a string of terminals and A is a non-terminal symbol. It determines whether the string x can be derived from A using the rules of G, answering yes if so or no otherwise.
- Conjecture query - EQUIV(H) where H is a context-free grammar. It determines of they are equivalent(they generate the same set of terminal strings from the start symbol S). If so it replies yes, otherwise gives a counterexample. (This can also be replaced by sampling as in the previous section.)

The Learner L^{cf}

The restriction to grammars in Chomsky normal form and the Learner's knowledge of T and V imply that the L^{cf} can explicitly enumerate all the possible productions of G in time bounded by a polynomial in the cardinalities of T and V.

We begin with all the possible production rules, P. The main loop asks EQUIV(H) question for the grammar H with T, V, S and P. If equivalent, it halts with output H. Otherwise, it "diagnoses" the counterexample (say t) returned, which result in at least production rule being removed from P. this process is then repeated.

In particular, the set P will always contain the productions of G as a subset, so the grammar H will always generate a superset of the strings generated by G. Hence, the only type of counterexample is one that is generated by H but not G.

Diagnosis

- Based on Shapiro's diagnosis algorithm to find a parse tree for t.
- It then uses membership queries to check parts if the parse tree against the true grammar G.
- The algorithm is guaranteed to terminate because at least one incorrect rule is removed with each counterexample, and the initial set of rules is finite.

Time analysis of L^{cf}. Every counterexample causes the elimination from P of another production that is not in G, which means that after at most $|P|$ counterexamples, L^{cf} must find a grammar equivalent to G and halt. The processing of each counterexample may be done in time polynomial in $|P|$ and the length of the counterexample. Since $|P|$ is bounded by a polynomial in $|T|$ and $|V|$, this means that the running time of L^{cf} is bounded by a polynomial in $|T|$, $|V|$, and the maximum length of any counterexample provided during the run.

To diagnose the counterexample t, L^{cf} finds a parse tree exhibiting a derivation of t from S using the productions in P. The root of the parse tree is labelled by S. Suppose it has two sons, labelled A and B, and the frontier of the subtree rooted at A is t_1 and the frontier of the subtree rooted at B is t_2. Clearly, $t = t_1 \cdot t_2$.

L^{cf} then asks the question MEMBER(t_1, A). If the reply is *no*, then t_1 can be derived from A in H but not in G and L^{cf} recursively continues to diagnose the subtree of the parse tree rooted at A. If the answer is *yes*, then L^{cf} asks the question MEMBER(t_2, B). If the reply is *no*, it recursively continues to diagnose the subtree of the parse tree rooted at B. If the answer is *yes*, then we have the fact that in G, t_1 may be derived from A and t_2 from B, but t may not be derived from S. Hence the production $S \rightarrow AB$ cannot be in the grammar G, and we remove it from P and return.

The recursive calls must eventually find a production of the form $A \rightarrow BC$ to discard from P, or arrive at a node labeled by a nonterminal A with only one son, labelled by a terminal symbol a. In this case, we have just asked MEMBER(a, A) and found that A does not generate a in G, so the production $A \rightarrow a$ may be removed from P.

Additional Comments

L^{cf} vs. L*: The Lf learning algorithm is simpler than L* because L^{cf} doesn't use information about non-terminal sets or the "enriched" membership queries that are available to L*.

Feasibility of Queries: The membership questions are computationally feasible (solvable in polynomial time), but the equivalence query is **not generally decidable**. This means it's often impossible to build a practical, "minimally adequate Teacher" for this domain, which is why a stochastic (probabilistic) approach might be better.

Counterexample Length: Unlike DFAs, the shortest counterexample for a grammar is not guaranteed to be short. A grammar can be constructed where the smallest counterexample to an initial hypothesis is a very long string, making the learning process slow. Eg - Strings of 1's whose length is not a positive multiple of fac(n).

Grammar Form and Convergence: The use of a specific type of grammar (Chomsky normal form) ensures the initial set of production rules is of a manageable size. The learning process then converges by iteratively removing rules, creating a "decreasing chain of subsets" until it reaches the correct hypothesis.

Open Problems

Can we substantially reduce the number of membership queries used by L^*? Can we do without membership queries altogether for either the deterministic or the stochastic settings for regular sets and still infer them efficiently?

An interesting domain proposed by Phil Laird is that of propositional Horn sentences. (That is, propositional sentences in conjunctive normal form with at most one positive literal per clause.) The satisfiability and equivalence problems for these sentences can be solved in polynomial time, making them an attractive class [6].

A minimally adequate Teacher for propositional Horn sentences must answer two types of questions. A membership query consists of an assignment of truth-values to the (known) variables of the unknown sentence ϕ, and the Teacher replies according to whether this assignment satisfies ϕ. A conjecture consists of a Horn sentence ψ, and the Teacher tests the equivalence of ϕ and ψ and replies either with *yes* or an assignment of truth-values that satisfies one of the sentences but not the other. There is a polynomial-time algorithm (in the sizes of ϕ and ψ) to implement a minimally adequate Teacher in this domain.

An interesting open problem is whether there is a Learner L^h that can learn any propositional Horn sentence ϕ from any minimally adequate Teacher of propositional Horn sentences in time polynomial in the size of ϕ. (So far, I have only been able to find such an algorithm with some annoying additional restrictions on the Teacher.)